

---

**Algorithm 1** fire\_elements

---

**Require:** Set of elements  $E$ **Ensure:** Fires all elements in  $E$  according to rules and propagates changes to parents

```
1: for each  $e$  in  $E$  do
2:   if try_any_rule( $e$ ) succeeds then
3:     add  $e$  to changed elements
4:      $P \leftarrow$  parents of  $e$ 
5:     fire_elements( $P$ )
6:   end if
7: end for
```

---

---

**Algorithm 2** try\_any\_rule

---

**Require:** Element  $e$ **Ensure:** Returns True if any rule applies to  $e$ , False otherwise

```
1: if try_pie_rule( $e$ ) succeeds then
2:   return True
3: else if try_por_rule( $e$ ) succeeds then
4:   return True
5: else if try_pand_rule( $e$ ) succeeds then
6:   return True
7: else if try_pmake_rule( $e$ ) succeeds then
8:   return True
9: else if try_pbreak_rule( $e$ ) succeeds then
10:  return True
11: else if try_bpfulfill_rule( $e$ ) succeeds then
12:  return True
13: else if try_bpdeny_rule( $e$ ) succeeds then
14:  return True
15: else
16:  return False
17: end if
```

---

---

**Algorithm 3** try\_pie\_rule

---

**Require:** Element  $e$ **Ensure:** Returns True if PIE rule applied, False otherwise

```
1: if  $e$  exists and is a leaf and status( $e$ )  $\neq (\top, \perp)$  then
2:   set status( $e$ )  $\leftarrow (\top, \perp)$ 
3:   return True
4: else
5:   return False
6: end if
```

---

---

**Algorithm 4** try\_pand\_rule

---

**Require:** Element  $e$ **Ensure:** Returns True if PAND rule applied, False otherwise

```
1:  $and\_links \leftarrow$  all AND links from  $e$ 
2: if  $and\_links$  not empty and all children have status  $(\top, \perp)$  then
3:   set status( $e$ )  $\leftarrow (\top, \perp)$ 
4:   return True
5: else
6:   return False
7: end if
```

---

---

**Algorithm 5** try\_por\_rule

---

**Require:** Element  $e$ **Ensure:** Returns True if POR rule applied, False otherwise

```
1:  $or\_links \leftarrow$  all OR links from  $e$ 
2: if any child of  $or\_links$  has status  $(\top, \perp)$  then
3:   set status( $e$ )  $\leftarrow (\top, \perp)$ 
4:   return True
5: else
6:   return False
7: end if
```

---

---

**Algorithm 6** try\_pmake\_rule

---

**Require:** Quality  $q$ **Ensure:** Returns True if PMAKE rule applied, False otherwise

```
1:  $make\_links \leftarrow$  all MAKE links from  $q$ 
2: if any source of  $make\_links$  has status  $(\top, \perp)$  and status( $q$ ) =? then
3:   set status( $q$ )  $\leftarrow \top$ 
4:   return True
5: else
6:   return False
7: end if
```

---

---

**Algorithm 7** try\_pbroke\_rule

---

**Require:** Quality  $q$ **Ensure:** Returns True if PBREAK rule applied, False otherwise

```
1:  $break\_links \leftarrow$  all BREAK links from  $q$ 
2: if any source of  $break\_links$  has status  $(\top, \perp)$  and status( $q$ ) =? then
3:   set status( $q$ )  $\leftarrow \perp$ 
4:   return True
5: else
6:   return False
7: end if
```

---

---

**Algorithm 8** try\_bpfulfill\_rule

---

**Require:** Quality  $q$ **Ensure:** Returns True if BPFULFILL rule applied, False otherwise

```
1:  $make\_links \leftarrow$  all MAKE links from  $q$ 
2: if any source of  $make\_links$  has status  $(\top, \perp)$  and  $status(q) = \perp$  then
3:   set  $status(q) \leftarrow \top$ 
4:   for each BREAK link  $b$  from  $q$  with source status  $(\top, \perp)$  do
5:     for each refinement  $e$  of  $b$  do
6:       if  $status(e) = (\top, \perp)$  then
7:         set  $status(e) \leftarrow (\top, \top)$ 
8:       end if
9:     end for
10:  end for
11:  return True
12: else
13:  return False
14: end if
```

---

---

**Algorithm 9** try\_bpdeny\_rule

---

**Require:** Quality  $q$ **Ensure:** Returns True if BPDENY rule applied, False otherwise

```
1:  $break\_links \leftarrow$  all BREAK links from  $q$ 
2: if any source of  $break\_links$  has status  $(\top, \perp)$  and  $status(q) = \top$  then
3:   set  $status(q) \leftarrow \perp$ 
4:   for each MAKE link  $m$  from  $q$  with source status  $(\top, \perp)$  do
5:     for each refinement  $e$  of  $m$  do
6:       if  $status(e) = (\top, \perp)$  then
7:         set  $status(e) \leftarrow (\top, \top)$ 
8:       end if
9:     end for
10:  end for
11:  return True
12: else
13:  return False
14: end if
```

---